



AI AC LAB 17

NAGASANI TRINANDINI

2303A52071

BATCH - 32

```
In [ ]: import sys
import os

# Check if we're in a Colab environment and if a GPU is available
if 'google.colab' in sys.modules and 'COLAB_TPU_ADDR' not in os.environ:
    print('Installing cudf-cu12... This might take a few moments.')
    !pip install cudf-cu12 --extra-index-url=https://pypi.nvidia.com
    print("cuDF installation command executed.")
else:
    print('Not in Colab or GPU not detected. Skipping cudf installation.')
```

Installing cudf-cu12... This might take a few moments.
Looking in indexes: <https://pypi.org/simple>, <https://pypi.nvidia.com>
Requirement already satisfied: cudf-cu12 in /usr/local/lib/python3.12/dist-packages (26.2.1)
Requirement already satisfied: cachetools in /usr/local/lib/python3.12/dist-packages (from cudf-cu12) (6.2.6)
Requirement already satisfied: cuda-python<13.0,>=12.9.2 in /usr/local/lib/python3.12/dist-packages (from cudf-cu12) (12.9.4)
Requirement already satisfied: cuda-toolkit==12.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[nvcc,nvrtc]==12.*->cudf-cu12) (12.8.1)
Requirement already satisfied: cupy-cuda12x>=13.6.0 in /usr/local/lib/python3.12/dist-packages (from cudf-cu12) (14.0.1)
Requirement already satisfied: fsspec>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from cudf-cu12) (2025.3.0)
Requirement already satisfied: libcudf-cu12==26.2.* in /usr/local/lib/python3.12/dist-packages (from cudf-cu12) (26.2.1)
Requirement already satisfied: numba-cuda<0.23.0,>=0.22.2 in /usr/local/lib/python3.12/dist-packages (from numba-cuda[cu12]<0.23.0,>=0.22.2->cudf-cu12) (0.22.2)
Requirement already satisfied: numba<0.62.0,>=0.60.0 in /usr/local/lib/python3.12/dist-packages (from cudf-cu12) (0.60.0)
Requirement already satisfied: numpy<3.0,>=1.23 in /usr/local/lib/python3.12/dist-packages (from cudf-cu12) (2.0.2)
Requirement already satisfied: nvtx>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from cudf-cu12) (0.2.14)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from cudf-cu12) (26.0)
Requirement already satisfied: pandas<2.4.0,>=2.0 in /usr/local/lib/python3.12/dist-packages (from cudf-cu12) (2.2.2)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from cudf-cu12) (18.1.0)
Requirement already satisfied: pylibcudf-cu12==26.2.* in /usr/local/lib/python3.12/dist-packages (from cudf-cu12) (26.2.1)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from cudf-cu12) (13.9.4)
Requirement already satisfied: rmm-cu12==26.2.* in /usr/local/lib/python3.12/dist-packages (from cudf-cu12) (26.2.0)
Requirement already satisfied: typing_extensions>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from cudf-cu12) (4.15.0)
Requirement already satisfied: nvidia-cuda-nvcc-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[nvcc,nvrtc]==12.*->cudf-cu12) (12.8.93)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[nvcc,nvrtc]==12.*->cudf-cu12) (12.8.93)
Requirement already satisfied: libkvikio-cu12==26.2.* in /usr/local/lib/python3.12/dist-packages (from libcudf-cu12==26.2.*->cudf-cu12) (26.2.0)
Requirement already satisfied: librmm-cu12==26.2.* in /usr/local/lib/python3.12/dist-packages (from libcudf-cu12==26.2.*->cudf-cu12) (26.2.0)
Requirement already satisfied: nvidia-libnvcomp-cu12==5.1.0.21 in /usr/local/lib/python3.12/dist-packages (from libcudf-cu12==26.2.*->cudf-cu12) (5.1.0.21)
Requirement already satisfied: rapids-logger==0.2.* in /usr/local/lib/python3.12/dist-packages (from libcudf-cu12==26.2.*->cudf-cu12) (0.2.3)
Requirement already satisfied: cuda-bindings~=12.9.4 in /usr/local/lib/python

```

3.12/dist-packages (from cuda-python<13.0,>=12.9.2->cudf-cu12) (12.9.4)
Requirement already satisfied: cuda-pathfinder==1.*,>=1.3.3 in /usr/local/lib/p
ython3.12/dist-packages (from cupy-cuda12x>=13.6.0->cudf-cu12) (1.4.3)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/pyt
hon3.12/dist-packages (from numba<0.62.0,>=0.60.0->cudf-cu12) (0.43.0)
Requirement already satisfied: cuda-core<1.0.0,>=0.3.2 in /usr/local/lib/python
3.12/dist-packages (from numba-cuda<0.23.0,>=0.22.2->numba-cuda[cu12]<0.2
3.0,>=0.22.2->cudf-cu12) (0.3.2)
Requirement already satisfied: nvidia-cuda-runtime-cu12 in /usr/local/lib/pytho
n3.12/dist-packages (from numba-cuda[cu12]<0.23.0,>=0.22.2->cudf-cu12) (12.8.9
0)
Requirement already satisfied: nvidia-nvjitlink-cu12 in /usr/local/lib/python
3.12/dist-packages (from numba-cuda[cu12]<0.23.0,>=0.22.2->cudf-cu12) (12.8.93)
Requirement already satisfied: nvidia-cuda-cccl-cu12 in /usr/local/lib/python
3.12/dist-packages (from numba-cuda[cu12]<0.23.0,>=0.22.2->cudf-cu12) (12.9.27)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python
3.12/dist-packages (from pandas<2.4.0,>=2.0->cudf-cu12) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-p
ackages (from pandas<2.4.0,>=2.0->cudf-cu12) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dis
t-packages (from pandas<2.4.0,>=2.0->cudf-cu12) (2025.3)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python
3.12/dist-packages (from rich->cudf-cu12) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python
3.12/dist-packages (from rich->cudf-cu12) (2.19.2)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-pac
kages (from markdown-it-py>=2.2.0->rich->cudf-cu12) (0.1.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packa
ges (from python-dateutil>=2.8.2->pandas<2.4.0,>=2.0->cudf-cu12) (1.17.0)
cuDF installation command executed.

```

```

In [ ]: print('--- Country-wise Count ---')
        display(health_exp_df_cudf['Country'].value_counts())

```

```

--- Country-wise Count ---
Country
USA          51
Japan        51
Germany      50
Canada       44
Great Britain 43
France       35
Name: count, dtype: int64

```

```

In [ ]: import cudf
        import pandas as pd
        import seaborn as sns

        # Load the HealthExp dataset using seaborn as a pandas DataFrame
        health_exp_df_pd = sns.load_dataset('healthexp')

        # Convert the pandas DataFrame to a cuDF DataFrame
        health_exp_df_cudf = cudf.DataFrame(health_exp_df_pd)

```

```
# Display the first 5 rows of the cuDF DataFrame
display(health_exp_df_cudf.head())
```

	Year	Country	Spending_USD	Life_Expectancy
0	1970	Germany	252.311	70.6
1	1970	France	192.143	72.2
2	1970	Great Britain	123.993	71.9
3	1970	Japan	150.437	72.0
4	1970	USA	326.961	70.9

```
In [ ]: display(health_exp_df_cudf.to_string())
```

```
'      Year      Country  Spending_USD  Life_Expectancy\n0      1970      Germany      252.311      70.6\n1      1970      France      192.143      72.2\n2      1970  Great Britain      123.993      71.9\n3      1970      Japan      150.437      72.0\n4      1970      USA      326.961      70.9\n...\n      ...      ...      ...      ...
Germany      6938.983      81.1\n270  2020      France      5468.418
82.3\n271  2020  Great Britain      5018.700      80.4\n272  2020
Japan      4665.641      84.7\n273  2020      USA      11859.179
77.0\n\n[274 rows x 4 columns]'
```

```
In [ ]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

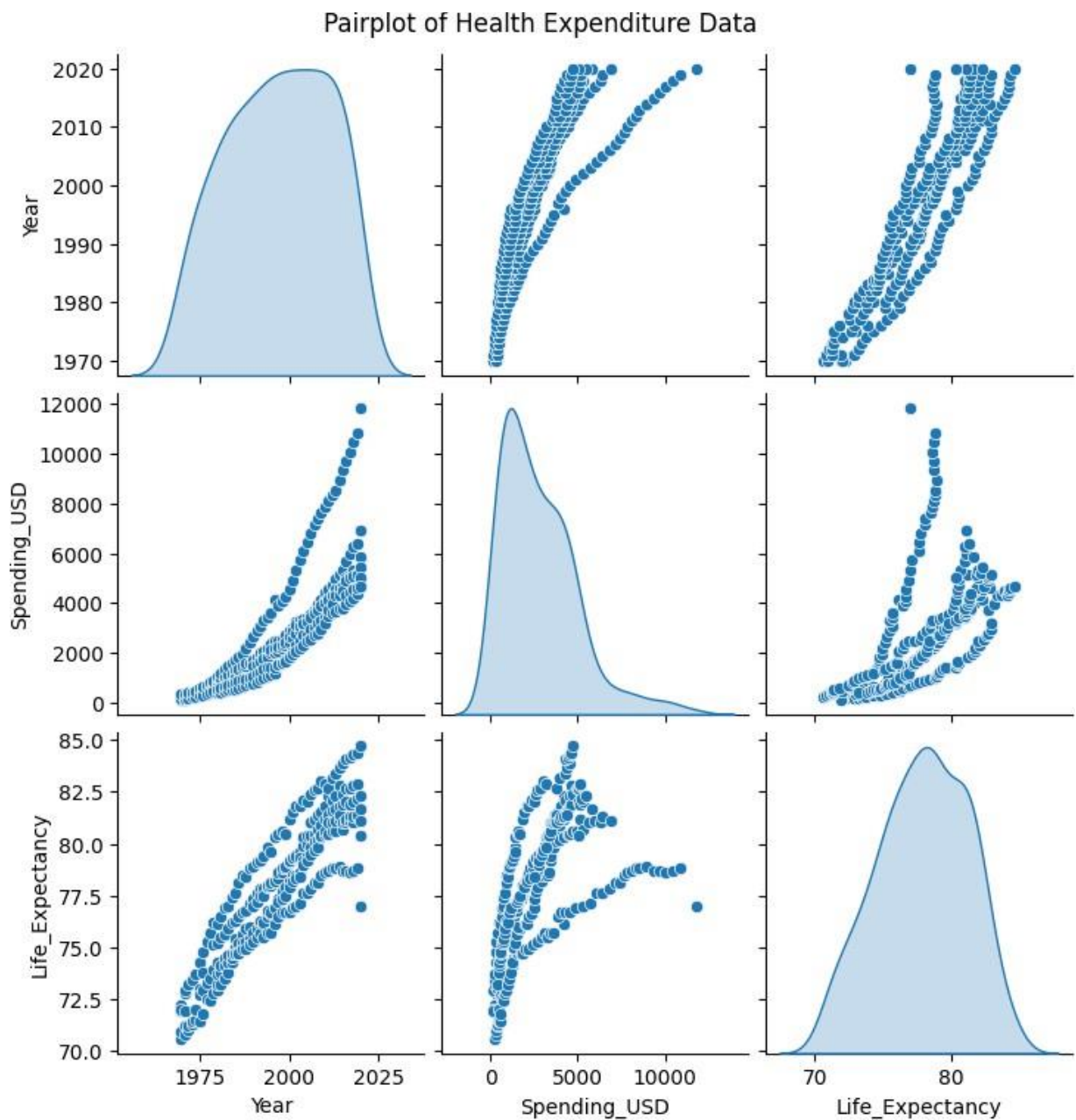
# Convert cuDF DataFrame to Pandas DataFrame for plotting
health_exp_df_pd = health_exp_df_cudf.to_pandas()

print('Converted cuDF to Pandas DataFrame for plotting.')
```

Converted cuDF to Pandas DataFrame for plotting.

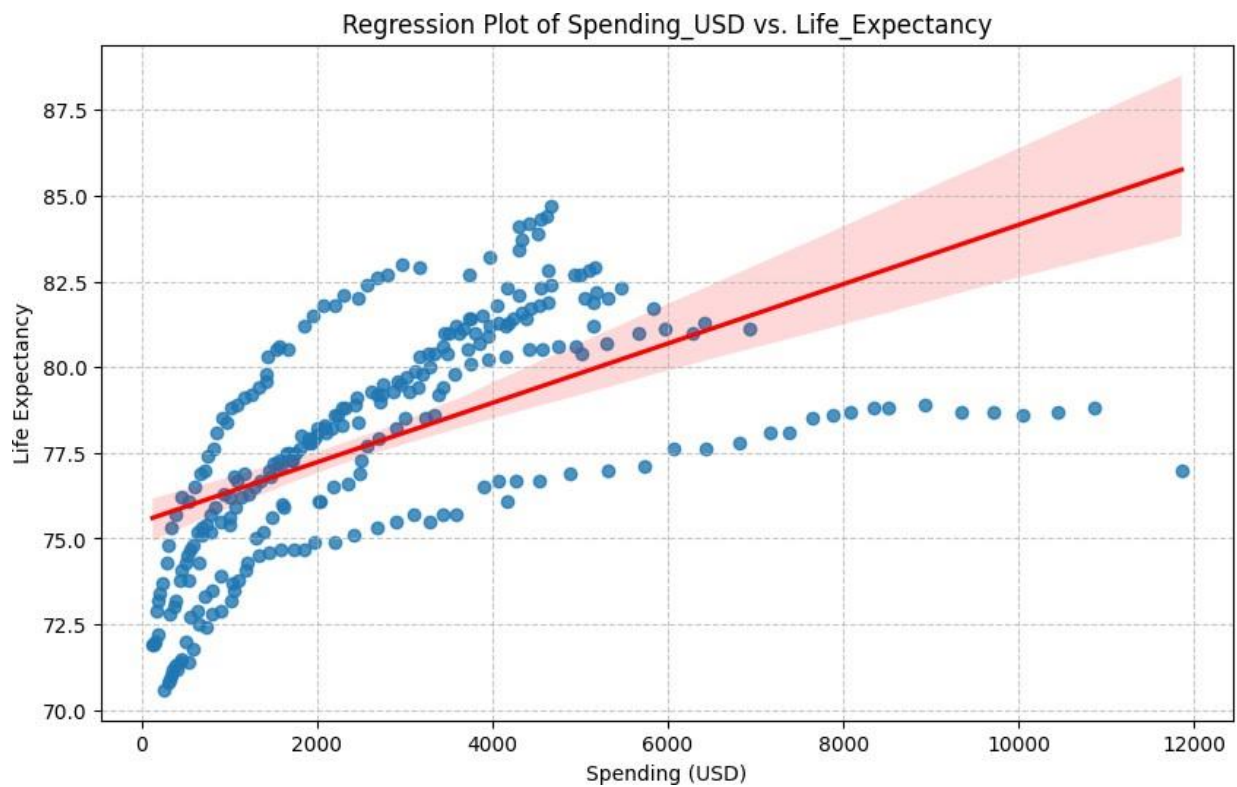
```
In [ ]: print('--- Generating Pairplot ---')
sns.pairplot(health_exp_df_pd, diag_kind='kde')
plt.suptitle('Pairplot of Health Expenditure Data', y=1.02) # Adjust suprtile
plt.show()
```

--- Generating Pairplot ---



```
In [ ]: print('--- Generating Regression Plot: Spending vs. Life Expectancy ---')
plt.figure(figsize=(10, 6))
sns.regplot(x='Spending_USD', y='Life_Expectancy', data=health_exp_df_pd, line
plt.title('Regression Plot of Spending_USD vs. Life_Expectancy')
plt.xlabel('Spending (USD)')
plt.ylabel('Life Expectancy')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

--- Generating Regression Plot: Spending vs. Life Expectancy ---

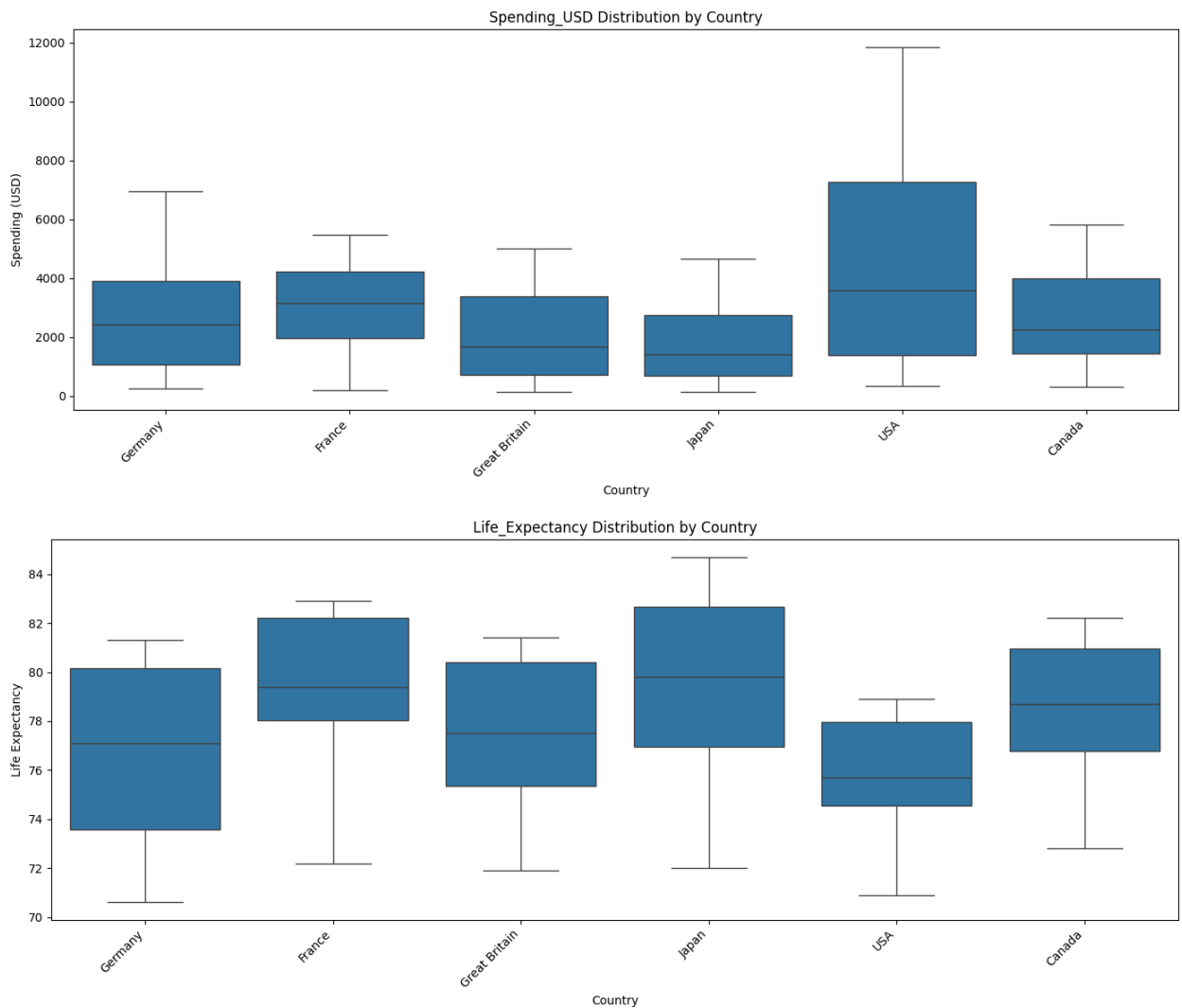


```
In [ ]: print('--- Generating Box Plots by Country ---')

plt.figure(figsize=(14, 6))
sns.boxplot(x='Country', y='Spending_USD', data=health_exp_df_pd)
plt.title('Spending_USD Distribution by Country')
plt.xlabel('Country')
plt.ylabel('Spending (USD)')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

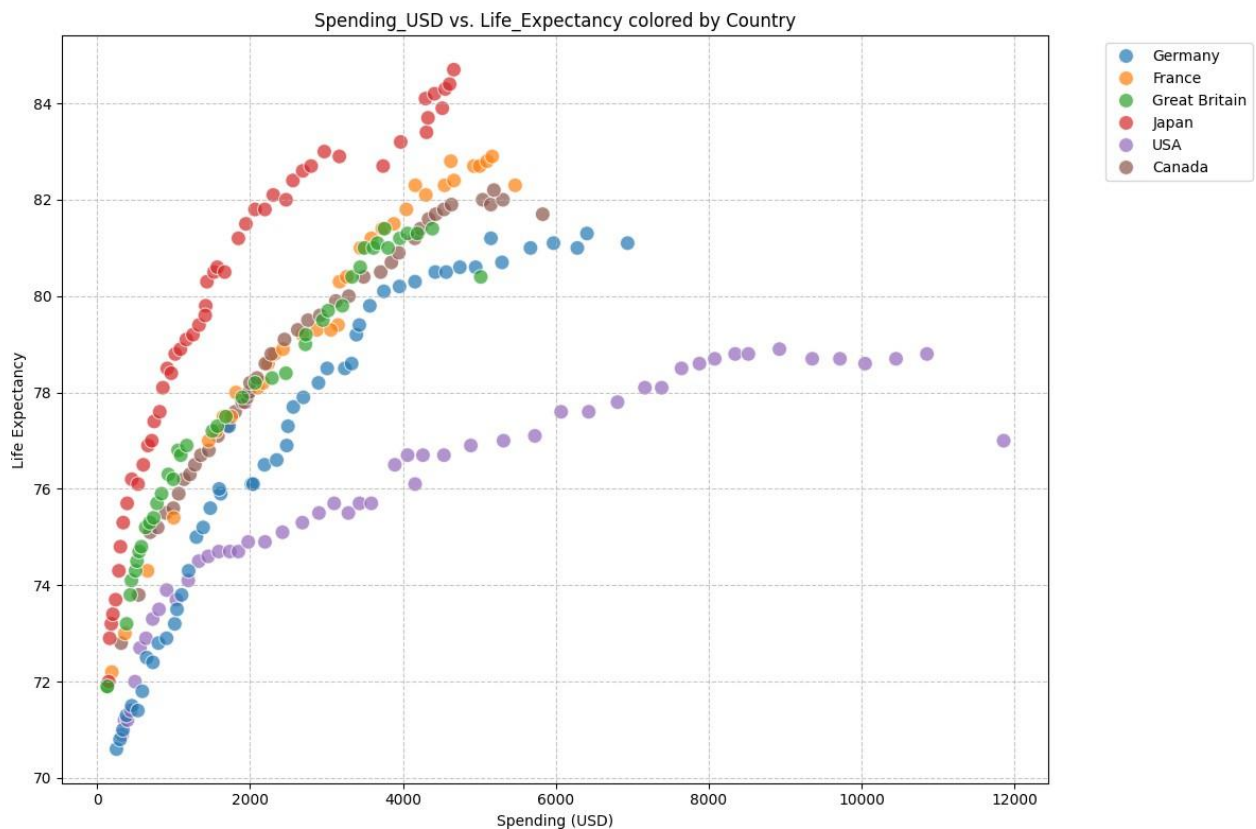
plt.figure(figsize=(14, 6))
sns.boxplot(x='Country', y='Life_Expectancy', data=health_exp_df_pd)
plt.title('Life_Expectancy Distribution by Country')
plt.xlabel('Country')
plt.ylabel('Life Expectancy')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

--- Generating Box Plots by Country ---



```
In [ ]: print('--- Generating Scatter Plot: Spending vs. Life Expectancy by Country --')
plt.figure(figsize=(12, 8))
sns.scatterplot(x='Spending_USD', y='Life_Expectancy', hue='Country', data=hea)
plt.title('Spending_USD vs. Life_Expectancy colored by Country')
plt.xlabel('Spending (USD)')
plt.ylabel('Life Expectancy')
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```

--- Generating Scatter Plot: Spending vs. Life Expectancy by Country ---



```
In [ ]: print('--- Missing Values ---')
missing_values = health_exp_df_cudf.isnull().sum()
# Filter out columns with no missing values for a cleaner display
missing_values = missing_values[missing_values > 0]

if not missing_values.empty:
    print(missing_values)
else:
    print('No missing values found in the dataset.')
```

```
--- Missing Values ---
No missing values found in the dataset.
```

```
In [ ]: import matplotlib.pyplot as plt
import seaborn as sns

# Convert the cudf Series to a pandas Series for plotting
missing_values_pd = missing_values.to_pandas() if not missing_values.empty else

if not missing_values_pd.empty:
    plt.figure(figsize=(10, 6))
    sns.barplot(x=missing_values_pd.index, y=missing_values_pd.values)
    plt.title('Missing Values per Column')
    plt.xlabel('Columns')
    plt.ylabel('Number of Missing Values')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()
```



```
else:
    print('No missing values to plot.')
```

No missing values to plot.

```
In [ ]: print('--- Column Information ---')
        health_exp_df_cudf.info()
```

```
--- Column Information ---
<class 'cudf.core.dataframe.DataFrame'>
RangeIndex: 274 entries, 0 to 273
Data columns (total 4 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Year                  274 non-null   int64
1   Country               274 non-null   object
2   Spending_USD          274 non-null   float64
3   Life_Expectancy       274 non-null   float64
dtypes: float64(2), int64(1), object(1)
memory usage: 9.2+ KB
```

```
In [ ]: print('\n--- Statistical Summary ---')
        display(health_exp_df_cudf.describe())
```

```
--- Statistical Summary ---
```

	Year	Spending_USD	Life_Expectancy
count	274.000000	274.000000	274.000000
mean	1996.992701	2789.338905	77.909489
std	14.180933	2194.939785	3.276263
min	1970.000000	123.993000	70.600000
25%	1985.250000	1038.357000	75.525000
50%	1998.000000	2295.578000	78.100000
75%	2009.000000	4055.610000	80.575000
max	2020.000000	11859.179000	84.700000



Set up Kaggle API Credentials

1. Go to [Kaggle](#).
2. Click on your profile picture in the top right corner and select "My Account".
3. Scroll down to the "API Tokens" section. Click on "New API Token". This will download a `kaggle.json` file to your computer.
4. In Google Colab, click on the folder icon on the left sidebar (File Browser).
5. Upload the `kaggle.json` file you just downloaded to the `/content/` directory.

```
In [ ]: # Install the Kaggle API client
!pip install kaggle

# Make sure kaggle.json is in the /content/ directory
# This moves the kaggle.json file to the required location and sets permission
!mkdir -p ~/.kaggle
!mv kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
```

```

Requirement already satisfied: kaggle in /usr/local/lib/python3.12/dist-packages (2.0.0)
Requirement already satisfied: bleach in /usr/local/lib/python3.12/dist-packages (from kaggle) (6.3.0)
Requirement already satisfied: kagglesdk<1.0,>=0.1.15 in /usr/local/lib/python3.12/dist-packages (from kaggle) (0.1.16)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from kaggle) (26.0)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from kaggle) (5.29.6)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.9.0.post0)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.12/dist-packages (from kaggle) (8.0.4)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kaggle) (4.67.3)
Requirement already satisfied: urllib3>=1.15.1 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.5.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packages (from bleach->kaggle) (0.5.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil->kaggle) (1.17.0)
Requirement already satisfied: text-unidecode>=1.3 in /usr/local/lib/python3.12/dist-packages (from python-slugify->kaggle) (1.3)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (3.4.6)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (3.11)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (2026.2.25)
mv: cannot stat 'kaggle.json': No such file or directory
chmod: cannot access '/root/.kaggle/kaggle.json': No such file or directory

```

```

In [ ]: # Download the dataset from Kaggle
# The Kaggle link provided was: https://www.kaggle.com/datasets/siddheshpujari
# The dataset slug is 'siddheshpujari/healthcare-risk-factors'
!kaggle datasets download -d siddheshpujari/healthcare-risk-factors

# Unzip the downloaded file
!unzip healthcare-risk-factors.zip

# Verify the contents (optional)
!ls

```

```

403 Client Error: Forbidden for url: https://api.kaggle.com/v1/datasets.Dataset
ApiService/GetDatasetMetadata
unzip: cannot find or open healthcare-risk-factors.zip, healthcare-risk-factors.zip.zip or healthcare-risk-factors.zip.ZIP.
sample_data

```

```

In [ ]: import pandas as pd

```

```
# Load the dataset into a pandas DataFrame
# Based on the content of the zip file, the main CSV file is healthcare-risk-f
df = pd.read_csv('/content/dirty_v3_path.csv')

# Display the first 5 rows of the DataFrame
display(df.head())
```

	Age	Gender	Medical Condition	Glucose	Blood Pressure	BMI	Oxygen Saturation	LengthOfStay
0	46.0	Male	Diabetes	137.04	135.27	28.90	96.04	6
1	22.0	Male	Healthy	71.58	113.27	26.29	97.54	2
2	50.0	NaN	Asthma	95.24	NaN	22.53	90.31	2
3	57.0	NaN	Obesity	NaN	130.53	38.47	96.60	5
4	66.0	Female	Hypertension	95.15	178.17	31.12	94.90	4

```
In [ ]: import numpy as np

# Calculate the percentage of missing values for the specified columns
missing_bp_percentage = df['Blood Pressure'].isnull().sum() / len(df) * 100
missing_chol_percentage = df['Cholesterol'].isnull().sum() / len(df) * 100

print(f"Percentage of missing values in 'Blood Pressure': {missing_bp_percenta
print(f"Percentage of missing values in 'Cholesterol': {missing_chol_percentag
```

Percentage of missing values in 'Blood Pressure': 15.00%

Percentage of missing values in 'Cholesterol': 0.00%

Since these are numerical columns, imputing with the median is a good strategy as it is less sensitive to outliers compared to the mean. Let's fill the missing values with the median of each respective column.

```
In [ ]: # Impute missing 'Blood Pressure' values with the median
median_bp = df['Blood Pressure'].median()
df['Blood Pressure'].fillna(median_bp, inplace=True)

# Impute missing 'Cholesterol' values with the median
median_chol = df['Cholesterol'].median()
df['Cholesterol'].fillna(median_chol, inplace=True)

# Verify that missing values have been handled
print("Missing values after imputation:")
print(df[['Blood Pressure', 'Cholesterol']].isnull().sum())
```

Missing values after imputation:

Blood Pressure 0

Cholesterol 0

dtype: int64

```
/tmp/ipykernel_3691/3889467487.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
```

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['Blood Pressure'].fillna(median_bp, inplace=True)
```

```
/tmp/ipykernel_3691/3889467487.py:7: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
```

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['Cholesterol'].fillna(median_chol, inplace=True)
```

The missing values in 'Blood Pressure' and 'Cholesterol' have been imputed using their respective medians.

```
In [ ]: import pandas as pd

# Identify categorical columns
categorical_cols = df.select_dtypes(include=['object', 'category']).columns

print(f"Categorical columns identified: {list(categorical_cols)}")

# Apply one-hot encoding with drop_first=True to avoid multicollinearity
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

# Display the first few rows of the encoded DataFrame and its shape
print("DataFrame after one-hot encoding:")
display(df_encoded.head())
print(f"Shape of DataFrame after encoding: {df_encoded.shape}")
```

```
Categorical columns identified: ['Gender', 'Medical Condition', 'random_notes']
DataFrame after one-hot encoding:
```

	Age	Glucose	Blood Pressure	BMI	Oxygen Saturation	LengthOfStay	Cholesterol	Triglyce
0	46.0	137.04	135.27	28.90	96.04	6	231.88	2
1	22.0	71.58	113.27	26.29	97.54	2	165.57	1
2	50.0	95.24	138.32	22.53	90.31	2	214.94	1
3	57.0	NaN	130.53	38.47	96.60	5	197.71	1
4	66.0	95.15	178.17	31.12	94.90	4	259.53	1

5 rows × 27 columns

Shape of DataFrame after encoding: (30000, 27)

```
In [ ]: from sklearn.preprocessing import StandardScaler

# Identify numerical columns (excluding the new one-hot encoded columns)
# We'll consider columns that are not boolean (from one-hot encoding) and not
numerical_cols = df_encoded.select_dtypes(include=['int64', 'float64']).column

print(f"Numerical columns identified for scaling: {list(numerical_cols)}")

# Initialize the StandardScaler
scaler = StandardScaler()

# Apply StandardScaler to the numerical columns
df_scaled = df_encoded.copy()
df_scaled[numerical_cols] = scaler.fit_transform(df_scaled[numerical_cols])

# Display the first few rows of the scaled DataFrame and its statistical summa
print("DataFrame after numerical feature scaling (first 5 rows):")
display(df_scaled.head())

print("Statistical summary of scaled numerical features:")
display(df_scaled[numerical_cols].describe())
```

Numerical columns identified for scaling: ['Age', 'Glucose', 'Blood Pressure', 'BMI', 'Oxygen Saturation', 'LengthOfStay', 'Cholesterol', 'Triglycerides', 'Hb Alc', 'Smoking', 'Alcohol', 'Physical Activity', 'Diet Score', 'Family History', 'Stress Level', 'Sleep Hours', 'noise_col']

DataFrame after numerical feature scaling (first 5 rows):

	Age	Glucose	Blood Pressure	BMI	Oxygen Saturation	LengthOfStay	Cholesterol
0	-0.539449	0.322729	-0.250163	0.073986	0.290409	0.574279	0.56224
1	-2.041956	-1.251732	-1.381416	-0.381614	0.691893	-0.874214	-1.41602
2	-0.289031	-0.682656	-0.093330	-1.037957	-1.243260	-0.874214	0.05686
3	0.149200	NaN	-0.493897	1.744520	0.440296	0.212156	-0.45716
4	0.712640	-0.684820	1.955780	0.461508	-0.014719	-0.149967	1.38714

5 rows × 27 columns

Statistical summary of scaled numerical features:

	Age	Glucose	Blood Pressure	BMI	Oxygen Saturation
count	2.550000e+04	2.550000e+04	3.000000e+04	3.000000e+04	3.000000e+04
mean	6.771054e-17	3.864795e-16	7.561359e-16	-2.188472e-16	2.663588e-15
std	1.000020e+00	1.000020e+00	1.000017e+00	1.000017e+00	1.000017e+00
min	-2.793209e+00	-2.484651e+00	-3.388361e+00	-3.631911e+00	-7.345814e+00
25%	-6.020535e-01	-6.576413e-01	-6.378742e-01	-6.783651e-01	-5.232652e-01
50%	2.399101e-02	-3.156181e-01	-9.333023e-02	-7.438943e-02	9.234343e-02
75%	7.126400e-01	3.123865e-01	5.396571e-01	5.819540e-01	6.490677e-01
max	2.152542e+00	4.687493e+00	4.434766e+00	4.952922e+00	4.045621e+00

```
In [ ]: # Impute missing 'Glucose' values with the median in df_scaled
median_glucose = df_scaled['Glucose'].median()
df_scaled['Glucose'].fillna(median_glucose, inplace=True)

# Verify that missing values in 'Glucose' have been handled
print("Missing values in 'Glucose' after imputation:")
print(df_scaled['Glucose'].isnull().sum())
```

Missing values in 'Glucose' after imputation:

0

/tmp/ipykernel_3691/2990116896.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df_scaled['Glucose'].fillna(median_glucose, inplace=True)
```

```
In [ ]: from sklearn.model_selection import train_test_split
```

```
# Define features (X) and a sample target variable (y)
# We'll use 'Medical Condition_Diabetes' as an example target.
# Remember to drop all one-hot encoded columns that are not your target from X

# Identify all one-hot encoded columns (assuming they are boolean or 'uint8')
one_hot_cols = df_scaled.select_dtypes(include=['bool', 'uint8']).columns.tolist()

# Define the target variable (y)
target_column = 'Medical Condition_Diabetes'
y = df_scaled[target_column]

# Define features (X) by dropping the target and other one-hot encoded columns
X = df_scaled.drop(columns=one_hot_cols)

# Split the dataset into training and testing sets (e.g., 80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Shape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")

# Display the first few rows of the training features and target
print("\nX_train head:")
display(X_train.head())

print("\ny_train head:")
display(y_train.head())
```

```
Shape of X_train: (24000, 17)
```

```
Shape of X_test: (6000, 17)
```

```
Shape of y_train: (24000,)
```

```
Shape of y_test: (6000,)
```

```
X_train head:
```


	Age	Glucose	Blood Pressure	BMI	Oxygen Saturation	LengthOfStay	Chole
289	-0.602054	-0.315618	0.257358	2.994365	0.761483	-0.512091	-0.4
27906	1.526498	-0.648742	-0.455845	-0.516025	0.595537	-0.149967	-0.8
3256	-1.415911	-0.305516	-0.093330	-0.303062	-2.292471	-0.512091	-1.1
5305	-1.228098	-0.586206	0.699061	-0.872126	0.368029	-0.149967	-1.7
741	1.088267	0.425432	0.388481	0.307896	0.857839	0.936403	0.1

y_train head:

Medical Condition_Diabetes	
289	False
27906	False
3256	False
5305	False
741	True

dtype: bool



To download datasets from Kaggle, you'll need a Kaggle API key. You can generate one from your Kaggle account settings

(<https://www.kaggle.com/<username>/account>). Look for the 'API' section and click 'Create New API Token'. This

Once you have the `kaggle.json` file, click on the Files icon (folder icon), then click 'Upload to session storage' icon. Upload your `kaggle.json` file there. Colab will prompt you that files uploaded to session storage are deleted when the runtime is recycled. Click 'OK'.

www.princexml.com

This document was created with Prince, a great way of getting web content onto paper.

Alternatively, you can upload the `kaggle.json` file to your Google Drive and mount your drive to access it. For simplicity, I'll assume you've uploaded it to the session storage at `/content/kaggle.json`.

```
In [ ]: import os

# Create a .kaggle directory if it doesn't exist
!mkdir -p ~/.kaggle

# Move the uploaded kaggle.json to the correct directory
# This assumes you've uploaded kaggle.json to /content/
!mv /content/kaggle.json ~/.kaggle/

# Set permissions
!chmod 600 ~/.kaggle/kaggle.json

print("Kaggle API key configured.")
```

```
mv: cannot stat '/content/kaggle.json': No such file or directory
chmod: cannot access '/root/.kaggle/kaggle.json': No such file or directory
Kaggle API key configured.
```

```
In [ ]: # Install the Kaggle library
!pip install kaggle
```

Requirement already satisfied: kaggle in /usr/local/lib/python3.12/dist-packages (2.0.0)

Requirement already satisfied: bleach in /usr/local/lib/python3.12/dist-packages (from kaggle) (6.3.0)

Requirement already satisfied: kagglesdk<1.0,>=0.1.15 in /usr/local/lib/python3.12/dist-packages (from kaggle) (0.1.16)

Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from kaggle) (26.0)

Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from kaggle) (5.29.6)

Requirement already satisfied: python-dateutil in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.9.0.post0)

Requirement already satisfied: python-slugify in /usr/local/lib/python3.12/dist-packages (from kaggle) (8.0.4)

Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.32.4)

Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kaggle) (4.67.3)

Requirement already satisfied: urllib3>=1.15.1 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.5.0)

Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packages (from bleach->kaggle) (0.5.1)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil->kaggle) (1.17.0)

Requirement already satisfied: text-unidecode>=1.3 in /usr/local/lib/python3.12/dist-packages (from python-slugify->kaggle) (1.3)

Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (3.4.6)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (3.11)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (2026.2.25)

```
In [ ]: # Download the dataset
# The dataset identifier is 'medaxone/top-100-social-media-profiles'
!kaggle datasets download -d medaxone/top-100-social-media-profiles

# Unzip the downloaded file
!unzip top-100-social-media-profiles.zip

print("Dataset downloaded and unzipped.")
```

Dataset URL: <https://www.kaggle.com/datasets/medaxone/top-100-social-media-profiles>

License(s): CC0-1.0

Downloading top-100-social-media-profiles.zip to /content

100% 8.92k/8.92k [00:00<00:00, 3.02MB/s]

Archive: top-100-social-media-profiles.zip

inflating: SocialMediaTop100.csv

Dataset downloaded and unzipped.

```
In [ ]: import pandas as pd
```

```
# Load the CSV file into a pandas DataFrame
# Correcting the filename and specifying 'latin1' encoding due to UnicodeDecod
csv_file_path = '/content/SocialMediaTop100.csv'
df = pd.read_csv(csv_file_path, encoding='latin1')

# Display the first 5 rows of the DataFrame
display(df.head())
```

	N	PROFILE	FOLLOWERS	POSTS	Platform
0	1	Instagram	539446645.0	7202.0	Instagram
1	2	Cristiano Ronaldo	473864939.0	3338.0	Instagram
2	3	Kylie <U+0001F90D>	364542529.0	6935.0	Instagram
3	4	Leo Messi	355790796.0	890.0	Instagram
4	5	Selena Gomez	341579063.0	1828.0	Instagram

```
In [ ]: # Display basic information about the DataFrame, including data types and non-
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0    N           500 non-null    int64
1    PROFILE     500 non-null    object
2    FOLLOWERS   500 non-null    float64
3    POSTS       400 non-null    float64
4    Platform    500 non-null    object
dtypes: float64(2), int64(1), object(2)
memory usage: 19.7+ KB
None
```

```
In [ ]: # Impute missing values in 'POSTS' with the median
median_posts = df['POSTS'].median()
df['POSTS'].fillna(median_posts, inplace=True)

# Convert 'POSTS' column to integer type
df['POSTS'] = df['POSTS'].astype(int)

# Clean 'PROFILE' and 'Platform' columns by stripping whitespace
df['PROFILE'] = df['PROFILE'].str.strip()
df['Platform'] = df['Platform'].str.strip()

print("Missing values in 'POSTS' imputed with median and converted to int. 'PR

# Display updated info and check for nulls again
print("\nUpdated DataFrame Info:")
print(df.info())
```

```
print("\nUpdated Missing Values:")
display(df.isnull().sum())
```

Missing values in 'POSTS' imputed with median and converted to int. 'PROFILE' and 'Platform' columns cleaned.

Updated DataFrame Info:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0    N           500 non-null    int64
1    PROFILE     500 non-null    object
2    FOLLOWERS   500 non-null    float64
3    POSTS       500 non-null    int64
4    Platform    500 non-null    object
dtypes: float64(1), int64(2), object(2)
memory usage: 19.7+ KB
None
```

Updated Missing Values:

/tmp/ipykernel_11788/1871419134.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['POSTS'].fillna(median_posts, inplace=True)
```

0

N 0

PROFILE 0

FOLLOWERS 0

POSTS 0

Platform 0

dtype: int64

```
In [ ]: # Impute missing values in 'POSTS' with the median
median_posts = df['POSTS'].median()
df['POSTS'].fillna(median_posts, inplace=True)
```

```

# Convert 'POSTS' column to integer type
df['POSTS'] = df['POSTS'].astype(int)

# Clean 'PROFILE' and 'Platform' columns by stripping whitespace
df['PROFILE'] = df['PROFILE'].str.strip()
df['Platform'] = df['Platform'].str.strip()

print("Missing values in 'POSTS' imputed with median and converted to int. 'PR

# Display updated info and check for nulls again
print("\nUpdated DataFrame Info:")
print(df.info())

print("\nUpdated Missing Values:")
display(df.isnull().sum())

```

Missing values in 'POSTS' imputed with median and converted to int. 'PROFILE' and 'Platform' columns cleaned.

Updated DataFrame Info:

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
----  -----
0    N           500 non-null    int64
1    PROFILE     500 non-null    object
2    FOLLOWERS   500 non-null    float64
3    POSTS       500 non-null    int64
4    Platform    500 non-null    object
dtypes: float64(1), int64(2), object(2)
memory usage: 19.7+ KB
None

```

Updated Missing Values:

/tmp/ipykernel_11788/1871419134.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['POSTS'].fillna(median_posts, inplace=True)
```

	0
N	0
PROFILE	0
FOLLOWERS	0
POSTS	0
Platform	0

dtype: int64

```
In [ ]: # Display descriptive statistics for numerical columns
display(df.describe())
```

	N	FOLLOWERS	POSTS
count	500.000000	5.000000e+02	400.000000
mean	50.500000	5.055225e+07	18537.182500
std	28.894979	6.100050e+07	66647.134154
min	1.000000	1.279972e+06	47.000000
25%	25.750000	2.440000e+07	826.250000
50%	50.500000	3.545000e+07	2470.500000
75%	75.250000	5.810649e+07	8211.750000
max	100.000000	5.394466e+08	892566.000000

```
In [ ]: # Check for missing values in each column
display(df.isnull().sum())
```

	0
N	0
PROFILE	0
FOLLOWERS	0
POSTS	100
Platform	0

dtype: int64

```
In [ ]: # Check for duplicate profiles before removal
initial_rows = df.shape[0]
duplicate_profiles = df[df.duplicated(subset=['PROFILE'])].shape[0]
```

```

print(f"Initial number of rows: {initial_rows}")
print(f"Number of duplicate profiles found: {duplicate_profiles}")

# Remove duplicate profiles based on the 'PROFILE' column, keeping the first o
df.drop_duplicates(subset=['PROFILE'], inplace=True)

# Verify the number of rows after removing duplicates
final_rows = df.shape[0]
print(f"Number of rows after removing duplicate profiles: {final_rows}")

# Display the first few rows of the DataFrame to show the change
display(df.head())

```

Initial number of rows: 500

Number of duplicate profiles found: 48

Number of rows after removing duplicate profiles: 452

	N	PROFILE	FOLLOWERS	POSTS	Platform
0	1	Instagram	539446645.0	7202	Instagram
1	2	Cristiano Ronaldo	473864939.0	3338	Instagram
2	3	Kylie <U+0001F90D>	364542529.0	6935	Instagram
3	4	Leo Messi	355790796.0	890	Instagram
4	5	Selena Gomez	341579063.0	1828	Instagram

```

In [ ]: median_posts = df['POSTS'].median()
df['POSTS'] = df['POSTS'].fillna(median_posts)

df['POSTS'] = df['POSTS'].astype(int)

df['PROFILE'] = df['PROFILE'].str.strip()
df['Platform'] = df['Platform'].str.strip()

print("Missing values in 'POSTS' imputed with median and converted to int. 'PR

print("\nUpdated DataFrame Info:")
print(df.info())

print("\nUpdated Missing Values:")
display(df.isnull().sum())

```


Missing values in 'POSTS' imputed with median and converted to int. 'PROFILE' and 'Platform' columns cleaned.

Updated DataFrame Info:

```
<class 'pandas.core.frame.DataFrame'>
```

Index: 452 entries, 0 to 499

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	N	452 non-null	int64
1	PROFILE	452 non-null	object
2	FOLLOWERS	452 non-null	float64
3	POSTS	452 non-null	int64
4	Platform	452 non-null	object

dtypes: float64(1), int64(2), object(2)

memory usage: 21.2+ KB

None

Updated Missing Values:

	0
N	0
PROFILE	0
FOLLOWERS	0
POSTS	0
Platform	0

dtype: int64

```
In [ ]: print("Unique values in 'Platform' before encoding:")
        print(df['Platform'].unique())

        # Perform one-hot encoding on the 'Platform' column
        df = pd.get_dummies(df, columns=['Platform'], drop_first=True)

        print("\nDataFrame after one-hot encoding 'Platform':")
        display(df.head())
        print("\nUpdated DataFrame Info:")
        print(df.info())
```

Unique values in 'Platform' before encoding:

```
['Instagram' 'Twitter' 'YouTube' 'Twitch' 'TikTok']
```

DataFrame after one-hot encoding 'Platform':

	N	PROFILE	FOLLOWERS	POSTS	Platform_TikTok	Platform_Twitch	Platform
0	1	Instagram	539446645.0	7202	False	False	
1	2	Cristiano Ronaldo	473864939.0	3338	False	False	
2	3	Kylie <U+0001F90D>	364542529.0	6935	False	False	
3	4	Leo Messi	355790796.0	890	False	False	
4	5	Selena Gomez	341579063.0	1828	False	False	

Updated DataFrame Info:

<class 'pandas.core.frame.DataFrame'>

Index: 452 entries, 0 to 499

Data columns (total 8 columns):

#	Column	Non-Null Count	Dtype
0	N	452 non-null	int64
1	PROFILE	452 non-null	object
2	FOLLOWERS	452 non-null	float64
3	POSTS	452 non-null	int64
4	Platform_TikTok	452 non-null	bool
5	Platform_Twitch	452 non-null	bool
6	Platform_Twitter	452 non-null	bool
7	Platform_YouTube	452 non-null	bool

dtypes: bool(4), float64(1), int64(2), object(1)

memory usage: 19.4+ KB

None



Important: Set Up Your Kaggle API Key in Colab Secrets

To download datasets from Kaggle, you need to provide your Kaggle API credentials. Please follow these steps:

1. Obtain your Kaggle

- Go to [Kaggle](https://www.kaggle.com).
- Log in or [register](#).
- Click on your profile picture in the top right corner and select "My Profile".
- Navigate to the "Account" tab.
- Under the "API" section, click "Create New API Token" to download `kaggle.json`.

2. Add `KAGGLE_USERNAME` and `KAGGLE_KEY` to Colab Secrets:

- In Google Colab, click on the "Secrets" icon in the left sidebar.
- Click "Add new secret" and create a secret named `KAGGLE_USERNAME`. Paste your Kaggle username (from `kaggle.json`) as its value.
- Click "Add new secret" again and create a secret named `KAGGLE_KEY`. Paste your Kaggle API key (from `kaggle.json`) as its value.
- **Ensure both secrets are enabled** by checking the toggle next to them.

If these steps are not correctly followed, the dataset download will fail.

```
In [21]: # Install the Kaggle API client
!pip install kaggle
```

Requirement already satisfied: kaggle in /usr/local/lib/python3.12/dist-packages (2.0.0)

Requirement already satisfied: bleach in /usr/local/lib/python3.12/dist-packages (from kaggle) (6.3.0)

Requirement already satisfied: kagglesdk<1.0,>=0.1.15 in /usr/local/lib/python3.12/dist-packages (from kaggle) (0.1.16)

Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from kaggle) (26.0)

Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from kaggle) (5.29.6)

Requirement already satisfied: python-dateutil in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.9.0.post0)

Requirement already satisfied: python-slugify in /usr/local/lib/python3.12/dist-packages (from kaggle) (8.0.4)

Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.32.4)

Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kaggle) (4.67.3)

Requirement already satisfied: urllib3>=1.15.1 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.5.0)

Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packages (from bleach->kaggle) (0.5.1)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil->kaggle) (1.17.0)

Requirement already satisfied: text-unidecode>=1.3 in /usr/local/lib/python3.12/dist-packages (from python-slugify->kaggle) (1.3)

Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (3.4.6)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (3.11)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (2026.2.25)

```
In [33]: # Check for duplicate rows
# Drop the 'Genre_list' column if it exists, as it contains unhashable list ob
if 'Genre_list' in df.columns:
    df_temp = df.drop(columns=['Genre_list'])
else:
    df_temp = df.copy()

duplicate_rows = df_temp.duplicated().sum()
print(f"Number of duplicate rows before dropping: {duplicate_rows}")

# Drop duplicate rows if any exist
if duplicate_rows > 0:
    df_temp.drop_duplicates(inplace=True)
    print(f"Number of duplicate rows after dropping: {df_temp.duplicated().sum}")
    print("Duplicate rows have been removed.")
else:
    print("No duplicate rows found.")

# If duplicates were dropped from df_temp, update the original df
# (This logic assumes you want to apply the deduplication to df, not just df_t
if 'Genre_list' in df.columns:
```

```

# If the Genre_list column exists, we need to re-merge it or handle it separately
# For simplicity, we'll keep the original df as is for now if Genre_list is not the cause of error
# and only deduplicate if Genre_list was not the cause of error in a different block
# However, to fix the specific error, we operate on df_temp.
print("Note: Duplicates check was performed on a temporary DataFrame without 'Genre_list'")
print("If you wish to remove duplicates from the original 'df', you need to decide how to handle 'Genre_list'")
print("For now, 'df' remains unchanged by this duplicate removal block.")
else:
    df = df_temp.copy() # Apply changes back to df if Genre_list was not an issue

# Display the shape of the DataFrame after cleaning (referring to df_temp for shape)
print(f"\nDataFrame shape after cleaning: {df_temp.shape}")

```

Number of duplicate rows before dropping: 0

No duplicate rows found.

Note: Duplicates check was performed on a temporary DataFrame without 'Genre_list'.

If you wish to remove duplicates from the original 'df', you need to decide how to handle 'Genre_list'.

For now, 'df' remains unchanged by this duplicate removal block.

DataFrame shape after cleaning: (1000, 12)

In [34]: **from** sklearn.preprocessing **import** MinMaxScaler

```

# Initialize MinMaxScaler
scaler = MinMaxScaler(feature_range=(0, 1))

# Reshape the 'Rating' column to a 2D array, as MinMaxScaler expects 2D input
df['Rating_scaled'] = scaler.fit_transform(df[['Rating']])

print("Original 'Rating' column statistics:")
display(df['Rating'].describe())

print("\nScaled 'Rating_scaled' column statistics:")
display(df['Rating_scaled'].describe())

```

Original 'Rating' column statistics:

	Rating
count	1000.000000
mean	6.723200
std	0.945429
min	1.900000
25%	6.200000
50%	6.800000
75%	7.400000
max	9.000000

dtype: float64

Scaled 'Rating_scaled' column statistics:

	Rating_scaled
count	1000.000000
mean	0.679324
std	0.133159
min	0.000000
25%	0.605634
50%	0.690141
75%	0.774648
max	1.000000

dtype: float64

```
In [31]: from sklearn.preprocessing import MultiLabelBinarizer
import pandas as pd

# Split genres into lists
df['Genre_list'] = df['Genre'].apply(lambda x: x.split(','))

# Initialize MultiLabelBinarizer
mlb = MultiLabelBinarizer()

# Fit and transform the genre lists
genre_encoded = mlb.fit_transform(df['Genre_list'])

# Create a DataFrame from the encoded genres with appropriate column names
genre_df = pd.DataFrame(genre_encoded, columns=mlb.classes_)
```

```
# Concatenate the new genre DataFrame with the original DataFrame
df_encoded = pd.concat([df, genre_df], axis=1)

# Drop the original 'Genre' column and the intermediate 'Genre_list' column
df_encoded = df_encoded.drop(columns=['Genre', 'Genre_list'])

print("DataFrame with encoded genres (first 5 rows):")
display(df_encoded.head())
```

DataFrame with encoded genres (first 5 rows):

	Rank	Title	Description	Director	Actors	Year	Runtime (Minutes)
0	1	Guardians of the Galaxy	A group of intergalactic criminals are forced ...	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...	2014	121
1	2	Prometheus	Following clues to the origin of mankind, a te...	Ridley Scott	Noomi Rapace, Logan Marshall-Green, Michael Fa...	2012	124
2	3	Split	Three girls are kidnapped by a man with a diag...	M. Night Shyamalan	James McAvoy, Anya Taylor-Joy, Haley Lu Richar...	2016	117
3	4	Sing	In a city of humanoid animals, a hustling thea...	Christophe Lourdelet	Matthew McConaughey, Reese Witherspoon, Seth Ma...	2016	108
4	5	Suicide Squad	A secret government agency recruits some of th...	David Ayer	Will Smith, Jared Leto, Margot Robbie, Viola D...	2016	123

5 rows × 31 columns

```
In [30]: # Impute missing values for 'Revenue (Millions)' with the mean
df['Revenue (Millions)'] = df['Revenue (Millions)'].fillna(df['Revenue (Millio

# Impute missing values for 'Metascore' with the mean
df['Metascore'] = df['Metascore'].fillna(df['Metascore'].mean())

print("Missing values after imputation:")
print(df.isnull().sum())
```

Missing values after imputation:

Rank	0
Title	0
Genre	0
Description	0
Director	0
Actors	0
Year	0
Runtime (Minutes)	0
Rating	0
Votes	0
Revenue (Millions)	0
Metascore	0

dtype: int64

```
In [29]: # Check for duplicate rows
duplicate_rows = df.duplicated().sum()
print(f"Number of duplicate rows before dropping: {duplicate_rows}")

# Drop duplicate rows if any exist
if duplicate_rows > 0:
    df.drop_duplicates(inplace=True)
    print(f"Number of duplicate rows after dropping: {df.duplicated().sum()}")
    print("Duplicate rows have been removed.")
else:
    print("No duplicate rows found.")

# Display the shape of the DataFrame after cleaning
print(f"\nDataFrame shape after cleaning: {df.shape}")
```

Number of duplicate rows before dropping: 0

No duplicate rows found.

DataFrame shape after cleaning: (1000, 12)

The new Kaggle link points to the dataset [PromptCloudHQ/imdb-data](https://www.kaggle.com/datasets/promptcloudhq/imdb-data) . Let's download and extract it.

```
In [24]: print("DataFrame Information:")
df.info()
```



```
DataFrame Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Rank                                  1000 non-null   int64
1   Title                                1000 non-null   object
2   Genre                                1000 non-null   object
3   Description                           1000 non-null   object
4   Director                             1000 non-null   object
5   Actors                               1000 non-null   object
6   Year                                  1000 non-null   int64
7   Runtime (Minutes)                    1000 non-null   int64
8   Rating                               1000 non-null   float64
9   Votes                                1000 non-null   int64
10  Revenue (Millions)                    872 non-null    float64
11  Metascore                            936 non-null    float64
dtypes: float64(3), int64(4), object(5)
memory usage: 93.9+ KB
```

```
In [25]: print("\nDataFrame Description (Numerical Columns):")
display(df.describe())
```

DataFrame Description (Numerical Columns):

	Rank	Year	Runtime (Minutes)	Rating	Votes	Reve (Millio
count	1000.000000	1000.000000	1000.000000	1000.000000	1.000000e+03	872.000
mean	500.500000	2012.783000	113.172000	6.723200	1.698083e+05	82.956
std	288.819436	3.205962	18.810908	0.945429	1.887626e+05	103.253
min	1.000000	2006.000000	66.000000	1.900000	6.100000e+01	0.000
25%	250.750000	2010.000000	100.000000	6.200000	3.630900e+04	13.270
50%	500.500000	2014.000000	111.000000	6.800000	1.107990e+05	47.985
75%	750.250000	2016.000000	123.000000	7.400000	2.399098e+05	113.715
max	1000.000000	2016.000000	191.000000	9.000000	1.791916e+06	936.630

```
In [26]: print("DataFrame Information:")
df.info()
```

```
DataFrame Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Rank                                  1000 non-null   int64
1   Title                                1000 non-null   object
2   Genre                                1000 non-null   object
3   Description                           1000 non-null   object
4   Director                             1000 non-null   object
5   Actors                               1000 non-null   object
6   Year                                  1000 non-null   int64
7   Runtime (Minutes)                    1000 non-null   int64
8   Rating                               1000 non-null   float64
9   Votes                                1000 non-null   int64
10  Revenue (Millions)                   872 non-null    float64
11  Metascore                            936 non-null    float64
dtypes: float64(3), int64(4), object(5)
memory usage: 93.9+ KB
```

```
In [27]: print("\nDataFrame Description (Numerical Columns):")
         display(df.describe())
```

DataFrame Description (Numerical Columns):

	Rank	Year	Runtime (Minutes)	Rating	Votes	Reve (Millio
count	1000.000000	1000.000000	1000.000000	1000.000000	1.000000e+03	872.000
mean	500.500000	2012.783000	113.172000	6.723200	1.698083e+05	82.956
std	288.819436	3.205962	18.810908	0.945429	1.887626e+05	103.253
min	1.000000	2006.000000	66.000000	1.900000	6.100000e+01	0.000
25%	250.750000	2010.000000	100.000000	6.200000	3.630900e+04	13.270
50%	500.500000	2014.000000	111.000000	6.800000	1.107990e+05	47.985
75%	750.250000	2016.000000	123.000000	7.400000	2.399098e+05	113.715
max	1000.000000	2016.000000	191.000000	9.000000	1.791916e+06	936.630

```
In [22]: # Download the dataset
         !kaggle datasets download -d PromptCloudHQ/imdb-data

         # Unzip the downloaded file
         # The name of the zip file will be 'imdb-data.zip' by default for this dataset
         !unzip -o imdb-data.zip

         print("Dataset download and unzip process started.")
```

Dataset URL: <https://www.kaggle.com/datasets/PromptCloudHQ/imdb-data>
License(s): other
Downloading imdb-data.zip to /content
100% 134k/134k [00:00<00:00, 66.5MB/s]

Archive: imdb-data.zip
 inflating: IMDB-Movie-Data.csv
Dataset download and unzip process started.

```
In [23]: import pandas as pd

# List files to find the correct CSV name (optional, but good for verification)
!ls

# The dataset typically contains a CSV file, often named 'IMDB-Movie-Data.csv'
df = pd.read_csv('IMDB-Movie-Data.csv') # Adjust filename if different after '

# Display the first 5 rows of the DataFrame
display(df.head())
```

imdb-data.zip IMDB-Movie-Data.csv sample_data

Rank		Title	Genre	Description	Director	
0	1	Guardians of the Galaxy	Action,Adventure,Sci-Fi	A group of intergalactic criminals are forced ...	James Gunn	Chris Pr Diesel, B Cooper, Z
1	2	Prometheus	Adventure,Mystery,Sci-Fi	Following clues to the origin of mankind, a te...	Ridlev Scott	Noomi R Logan Ma Green, Micha
2	3	Split	Horror,Thriller	Three girls are kidnapped by a man with a diag...	M. Night Shyamalan	James McAvoy Taylor-Joy, H R
3	4	Sing	Animation,Comedy,Family	In a city of humanoid animals, a hustling thea...	Christophe Lourdelet	McConaughey Witherspoo M
4	5	Suicide Squad	Action,Adventure,Fantasy	A secret government agency recruits some of th...	David Ayer	Will Smith Leto, Margot R Vi

```
import torch
import torchvision
import torchvision.transforms as transforms

# Check if CUDA is available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

Using device: cuda

Next, we will define transformations for the dataset and then download the FashionMNIST dataset. Transformations typically involve converting images to tensors and normalizing them.

```
# Define transformations
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Download and load the FashionMNIST training dataset
trainset = torchvision.datasets.FashionMNIST(
    root='./data',
    train=True,
    download=True,
    transform=transform
)

# Download and load the FashionMNIST test dataset
testset = torchvision.datasets.FashionMNIST(
    root='./data',
    train=False,
    download=True,
    transform=transform
)

print(f"Training dataset size: {len(trainset)}")
print(f"Test dataset size: {len(testset)}")
```

Training dataset size: 60000
Test dataset size: 10000

Finally, we'll create data loaders for both the training and test sets. Data loaders help in iterating over the dataset in batches, which is efficient for training neural networks.

```
# Create data loaders
batch_size = 64

trainloader = torch.utils.data.DataLoader(
    trainset,
    batch_size=batch_size,
    shuffle=True,
    num_workers=2
)

testloader = torch.utils.data.DataLoader(
    testset,
    batch_size=batch_size,
    shuffle=False,
    num_workers=2
)

print(f"Number of batches in training loader: {len(trainloader)}")
print(f"Number of batches in test loader: {len(testloader)}")

# Example of getting a batch of data
dataiter = iter(trainloader)
images, labels = next(dataiter)

print(f"Shape of one batch of images: {images.shape}")
print(f"Shape of one batch of labels: {labels.shape}")
```

```

Number of batches in training loader: 938
Number of batches in test loader: 157
Shape of one batch of images: torch.Size([64, 1, 28, 28])
Shape of one batch of labels: torch.Size([64])

```

Dataset Information

Based on the previous steps, we have the following details about the FashionMNIST dataset:

- **Training dataset size:** 60,000 images
- **Test dataset size:** 10,000 images
- **Image dimensions:** Each image is 1 channel (grayscale) and 28x28 pixels.

Now, let's visualize a few sample images from the training dataset along with their corresponding class labels.

```

import matplotlib.pyplot as plt
import numpy as np

# Define the FashionMNIST class names
class_names = [
    'T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
    'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot'
]

# Get a batch of training images and labels (from the existing 'images' and 'labels' variables)
# Assuming 'images' and 'labels' from the last execution of the data loader cell are available.
# If not, uncomment the following two lines to get a new batch:
# dataiter = iter(trainloader)
# images, labels = next(dataiter)

# Denormalize images for display (undo the Normalize((0.5,), (0.5,)) transform)
# The original transform was: (image - mean) / std => (image - 0.5) / 0.5
# So, to reverse: image * std + mean => image * 0.5 + 0.5
images_denorm = images * 0.5 + 0.5 # Scale back to [0, 1] range

# Create a figure and a set of subplots
fig = plt.figure(figsize=(10, 10))

# Display up to 9 images from the batch
for i in range(9):
    ax = fig.add_subplot(3, 3, i + 1, xticks=[], yticks=[])

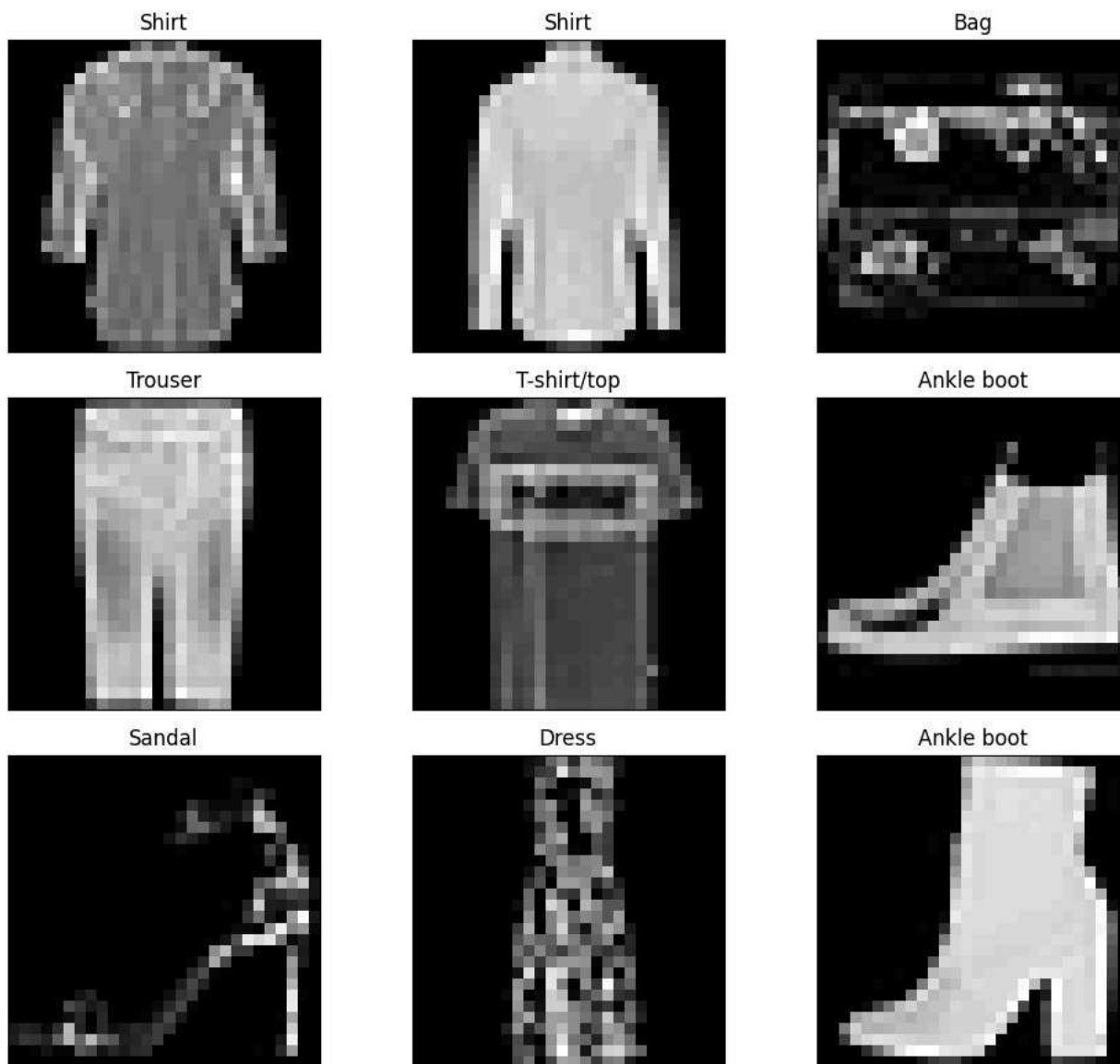
    # Convert tensor to numpy array and rearrange dimensions for matplotlib (C, H, W) -> (H, W, C)
    # For grayscale, it's (1, H, W) -> (H, W)
    plt.imshow(np.squeeze(images_denorm[i].numpy()), cmap='gray')

    # Set the title to the corresponding class name
    ax.set_title(class_names[labels[i]])

plt.suptitle('Sample FashionMNIST Images and Labels', fontsize=16)
plt.tight_layout(rect=[0, 0.03, 1, 0.95]) # Adjust layout to prevent title overlap
plt.show()

```

Sample FashionMNIST Images and Labels



Let's define a 5-layered Convolutional Neural Network (CNN) model. This model will consist of:

1. **Convolutional Layer 1:** Applies filters to extract features.
2. **Pooling Layer 1:** Reduces spatial dimensions.
3. **Convolutional Layer 2:** Further feature extraction.
4. **Pooling Layer 2:** Further reduces spatial dimensions.
5. **Fully Connected Layer 1:** Processes flattened features.
6. **Fully Connected Layer 2:** Another dense layer.
7. **Output Layer:** Produces class scores for the 10 FashionMNIST classes.

We'll use `ReLU` activation after each convolutional and fully connected layer (except the output layer) and `MaxPool2d` for pooling.

```
import torch.nn as nn
import torch.nn.functional as F

# Define the 5-layered CNN model
class FashionCNN(nn.Module):
    def __init__(self):
        super(FashionCNN, self).__init__()
        # Layer 1: Convolutional Layer 1
        # Input: 1 channel (grayscale), Output: 32 channels, Kernel size: 3x3
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1)
```

```

# Layer 2: Pooling Layer 1 (Max Pooling)
# Applies after ReLU activation, reduces 2x2
self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

# Layer 3: Convolutional Layer 2
# Input: 32 channels, Output: 64 channels, Kernel size: 3x3
self.conv2 = nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3, padding=1)

# Calculate output size after conv and pool layers
# Input image size: 28x28
# After conv1 (padding 1, stride 1) -> 28x28
# After pool (kernel 2, stride 2) -> 14x14
# After conv2 (padding 1, stride 1) -> 14x14
# After pool (kernel 2, stride 2) -> 7x7

# Layer 4: Fully Connected Layer 1
# Input: 64 channels * 7 * 7 (flattened output from conv/pool), Output: 128 neurons
self.fc1 = nn.Linear(64 * 7 * 7, 128)

# Layer 5: Fully Connected Layer 2
# Input: 128 neurons, Output: 64 neurons
self.fc2 = nn.Linear(128, 64)

# Output Layer: Fully Connected Layer 3 (for 10 classes)
# Input: 64 neurons, Output: 10 neurons (for 10 FashionMNIST classes)
self.fc3 = nn.Linear(64, 10)

def forward(self, x):
    # Conv Layer 1 -> ReLU -> Pool
    x = self.pool(F.relu(self.conv1(x)))
    # Conv Layer 2 -> ReLU -> Pool
    x = self.pool(F.relu(self.conv2(x)))

    # Flatten the output for the fully connected layers
    x = x.view(-1, 64 * 7 * 7) # -1 infers the batch size

    # Fully Connected Layer 1 -> ReLU
    x = F.relu(self.fc1(x))
    # Fully Connected Layer 2 -> ReLU
    x = F.relu(self.fc2(x))
    # Output Layer
    x = self.fc3(x)
    return x

# Instantiate the model and move it to the device (CPU or CUDA)
model = FashionCNN().to(device)

print("Model architecture:")
print(model)

```

```

Model architecture:
FashionCNN(
  (conv1): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (fc1): Linear(in_features=3136, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=64, bias=True)
  (fc3): Linear(in_features=64, out_features=10, bias=True)
)

```

First, we need to define the **Loss Function** and the **Optimizer**.

- **Loss Function:** Measures how well our model is doing with respect to the expected output. For classification tasks like FashionMNIST, `CrossEntropyLoss` is commonly used.
- **Optimizer:** An algorithm used to update the weights and biases of our neural network to minimize the loss function. `Adam` is a popular choice due to its efficiency.

```

import torch.optim as optim

# Define the loss function
criterion = nn.CrossEntropyLoss()

# Define the optimizer
optimizer = optim.Adam(model.parameters(), lr=0.001)

```

```
print("Loss Function: CrossEntropyLoss")
print("Optimizer: Adam with Learning Rate = 0.001")
```

```
Loss Function: CrossEntropyLoss
Optimizer: Adam with Learning Rate = 0.001
```

Now, let's implement the **Training Loop**. This loop will iterate over the training dataset for a specified number of epochs, performing the following steps for each batch:

1. **Forward Pass:** Input data through the model to get predictions.
2. **Calculate Loss:** Compare predictions to actual labels.
3. **Backward Pass:** Calculate gradients of the loss with respect to model parameters.
4. **Optimizer Step:** Update model parameters using the gradients.

We will also evaluate the model's performance on the test set after each epoch to monitor for overfitting and overall progress.

```
num_epochs = 10

train_losses = []
test_accuracies = []

print("Starting training...")

for epoch in range(num_epochs):
    model.train() # Set the model to training mode
    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        inputs, labels = data
        inputs, labels = inputs.to(device), labels.to(device) # Move data to GPU if available

        # Zero the parameter gradients
        optimizer.zero_grad()

        # Forward pass
        outputs = model(inputs)
        loss = criterion(outputs, labels)

        # Backward pass and optimize
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    avg_train_loss = running_loss / len(trainloader)
    train_losses.append(avg_train_loss)

    # Evaluate on test set
    model.eval() # Set the model to evaluation mode
    correct = 0
    total = 0
    with torch.no_grad(): # Disable gradient calculation during evaluation
        for data in testloader:
            images, labels = data
            images, labels = images.to(device), labels.to(device) # Move data to GPU if available
            outputs = model(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    accuracy = 100 * correct / total
    test_accuracies.append(accuracy)

    print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {avg_train_loss:.4f}, Test Accuracy: {accuracy:.2f}%')

print('\nFinished Training!')

# Optional: Plot training loss and test accuracy
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(range(1, num_epochs + 1), train_losses, marker='o')
plt.title('Training Loss per Epoch')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.grid(True)
```



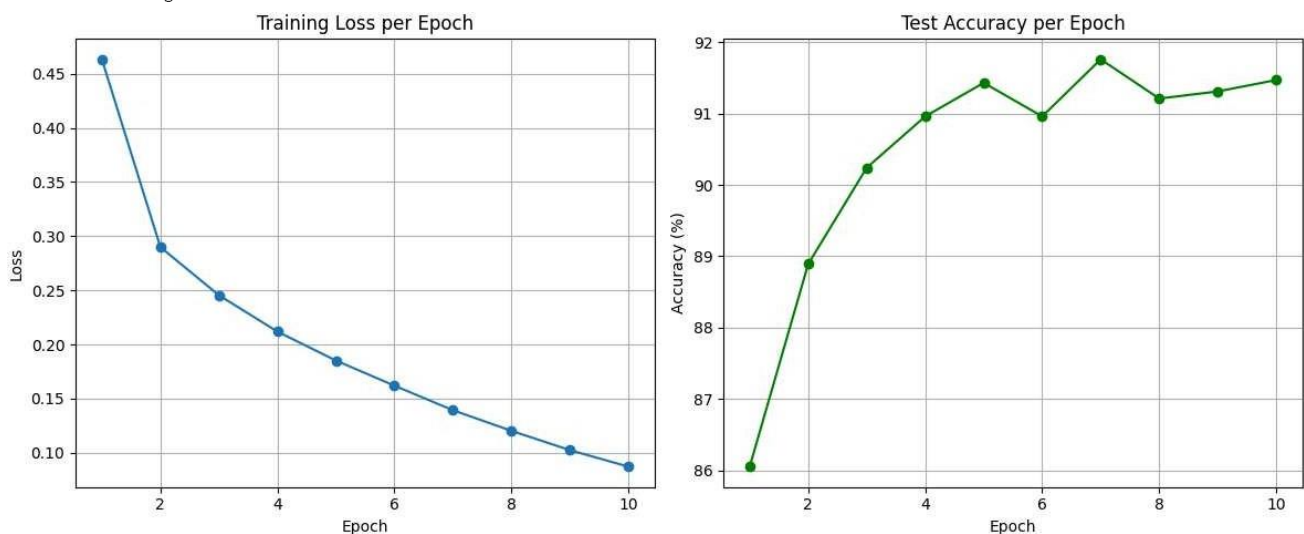
```
plt.subplot(1, 2, 2)
plt.plot(range(1, num_epochs + 1), test_accuracies, marker='o', color='green')
plt.title('Test Accuracy per Epoch')
plt.xlabel('Epoch')
plt.ylabel('Accuracy (%)')
plt.grid(True)

plt.tight_layout()
plt.show()
```

Starting training...

```
Epoch [1/10], Loss: 0.4632, Test Accuracy: 86.05%
Epoch [2/10], Loss: 0.2899, Test Accuracy: 88.89%
Epoch [3/10], Loss: 0.2455, Test Accuracy: 90.24%
Epoch [4/10], Loss: 0.2119, Test Accuracy: 90.96%
Epoch [5/10], Loss: 0.1852, Test Accuracy: 91.43%
Epoch [6/10], Loss: 0.1620, Test Accuracy: 90.96%
Epoch [7/10], Loss: 0.1394, Test Accuracy: 91.76%
Epoch [8/10], Loss: 0.1203, Test Accuracy: 91.21%
Epoch [9/10], Loss: 0.1025, Test Accuracy: 91.31%
Epoch [10/10], Loss: 0.0872, Test Accuracy: 91.47%
```

Finished Training!



✓ Designing an EfficientNet Model

Now, let's implement an EfficientNet model for our image classification task. We'll use a pre-trained `efficientnet_b0` model from `torchvision.models`, which is a common practice in transfer learning to leverage features learned from a large dataset like ImageNet. We will then modify the final classification layer to output 10 classes, corresponding to the FashionMNIST categories.

```
import torchvision.models as models

# Load a pre-trained EfficientNet-B0 model
efficientnet_model = models.efficientnet_b0(pretrained=True)

# Modify the classifier for 10 output classes (FashionMNIST)
num_fts = efficientnet_model.classifier[1].in_features
efficientnet_model.classifier[1] = nn.Linear(num_fts, 10)

# Move the model to the specified device
efficientnet_model = efficientnet_model.to(device)

print("EfficientNet model architecture after modification:")
print(efficientnet_model)
```

```
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated
warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `I
```

```
warnings.warn(msg)
Downloading: "https://download.pytorch.org/models/efficientnet_b0_rwightman-7f5810bc.pth" to /root/.cache/torch/hub/checkpoint
100%|██████████| 20.5M/20.5M [00:00<00:00, 149MB/s]
EfficientNet model architecture after modification:
EfficientNet(
  (features): Sequential(
    (0): Conv2dNormActivation(
      (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): SiLU(inplace=True)
    )
    (1): Sequential(
      (0): MBConv(
        (block): Sequential(
          (0): Conv2dNormActivation(
            (0): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), groups=32, bias=False)
            (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
            (2): SiLU(inplace=True)
          )
          (1): SqueezeExcitation(
            (avgpool): AdaptiveAvgPool2d(output_size=1)
            (fc1): Conv2d(32, 8, kernel_size=(1, 1), stride=(1, 1))
            (fc2): Conv2d(8, 32, kernel_size=(1, 1), stride=(1, 1))
            (activation): SiLU(inplace=True)
            (scale_activation): Sigmoid()
          )
          (2): Conv2dNormActivation(
            (0): Conv2d(32, 16, kernel_size=(1, 1), stride=(1, 1), bias=False)
            (1): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          )
        )
        (stochastic_depth): StochasticDepth(p=0.0, mode=row)
      )
    )
    (2): Sequential(
      (0): MBConv(
        (block): Sequential(
          (0): Conv2dNormActivation(
            (0): Conv2d(16, 96, kernel_size=(1, 1), stride=(1, 1), bias=False)
            (1): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
            (2): SiLU(inplace=True)
          )
          (1): Conv2dNormActivation(
            (0): Conv2d(96, 96, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), groups=96, bias=False)
            (1): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
            (2): SiLU(inplace=True)
          )
          (2): SqueezeExcitation(
            (avgpool): AdaptiveAvgPool2d(output_size=1)
            (fc1): Conv2d(96, 4, kernel_size=(1, 1), stride=(1, 1))
            (fc2): Conv2d(4, 96, kernel_size=(1, 1), stride=(1, 1))
            (activation): SiLU(inplace=True)
            (scale_activation): Sigmoid()
          )
        )
      )
    )
  )
)
```

✓ Training and Evaluation for EfficientNet Model

Now we will train the `efficientnet_model` using the same loss function (`CrossEntropyLoss`) and optimizer (`Adam`) as before. We will train it for 10 epochs and evaluate its performance on the test set after each epoch.

```
# Define the loss function for EfficientNet
efficientnet_criterion = nn.CrossEntropyLoss()

# Define the optimizer for EfficientNet
efficientnet_optimizer = optim.Adam(efficientnet_model.parameters(), lr=0.001)

print("EfficientNet Loss Function: CrossEntropyLoss")
print("EfficientNet Optimizer: Adam with Learning Rate = 0.001")
```

```
EfficientNet Loss Function: CrossEntropyLoss
EfficientNet Optimizer: Adam with Learning Rate = 0.001
```

```
num_epochs_efficientnet = 10

efficientnet_train_losses = []
efficientnet_test_accuracies = []

print("Starting EfficientNet training...")
```

```

# EfficientNet models typically expect 3-channel input images, so we need to adjust the input.
# FashionMNIST images are 1-channel (grayscale). We will replicate the single channel to 3 channels.
# Also, EfficientNet models are usually trained on larger images (e.g., 224x224), but we'll feed 28x28 for now.
# For proper transfer learning, you might want to resize images to 224x224 and normalize with ImageNet stats.
# For this demonstration, we'll keep the original size and replicate channels.

for epoch in range(num_epochs_efficientnet):
    efficientnet_model.train() # Set the model to training mode
    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        inputs, labels = data

        # Replicate grayscale channel to 3 channels for EfficientNet
        inputs = inputs.repeat(1, 3, 1, 1)

        inputs, labels = inputs.to(device), labels.to(device) # Move data to GPU if available

        # Zero the parameter gradients
        efficientnet_optimizer.zero_grad()

        # Forward pass
        outputs = efficientnet_model(inputs)
        loss = efficientnet_criterion(outputs, labels)

        # Backward pass and optimize
        loss.backward()
        efficientnet_optimizer.step()

        running_loss += loss.item()

    avg_train_loss = running_loss / len(trainloader)
    efficientnet_train_losses.append(avg_train_loss)

    # Evaluate on test set
    efficientnet_model.eval() # Set the model to evaluation mode
    correct = 0
    total = 0
    with torch.no_grad(): # Disable gradient calculation during evaluation
        for data in testloader:
            images, labels = data

            # Replicate grayscale channel to 3 channels for EfficientNet
            images = images.repeat(1, 3, 1, 1)

            images, labels = images.to(device), labels.to(device) # Move data to GPU if available
            outputs = efficientnet_model(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    accuracy = 100 * correct / total
    efficientnet_test accuracies.append(accuracy)

    print(f'EfficientNet Epoch [{epoch+1}/{num_epochs_efficientnet}], Loss: {avg_train_loss:.4f}, Test Accuracy: {accuracy:.2f}%')

print('\nFinished EfficientNet Training!')

# Plot training loss and test accuracy for EfficientNet
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(range(1, num_epochs_efficientnet + 1), efficientnet_train_losses, marker='o', color='blue')
plt.title('EfficientNet Training Loss per Epoch')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(range(1, num_epochs_efficientnet + 1), efficientnet_test accuracies, marker='o', color='green')
plt.title('EfficientNet Test Accuracy per Epoch')
plt.xlabel('Epoch')
plt.ylabel('Accuracy (%)')
plt.grid(True)

plt.tight_layout()
plt.show()

```

Starting EfficientNet training...

EfficientNet Epoch [1/10], Loss: 0.4637, Test Accuracy: 89.51%
EfficientNet Epoch [2/10], Loss: 0.2897, Test Accuracy: 90.94%
EfficientNet Epoch [3/10], Loss: 0.2472, Test Accuracy: 91.38%
EfficientNet Epoch [4/10], Loss: 0.2294, Test Accuracy: 90.97%
EfficientNet Epoch [5/10], Loss: 0.2092, Test Accuracy: 91.48%
EfficientNet Epoch [6/10], Loss: 0.1907, Test Accuracy: 92.78%
EfficientNet Epoch [7/10], Loss: 0.1705, Test Accuracy: 92.40%
EfficientNet Epoch [8/10], Loss: 0.1676, Test Accuracy: 91.23%
EfficientNet Epoch [9/10], Loss: 0.1645, Test Accuracy: 92.75%
EfficientNet Epoch [10/10], Loss: 0.1513, Test Accuracy: 92.83%

Finished EfficientNet Training!

